Fundamentos de Machine Learning con `scikit-learn`: Una Guía Práctica

Santiago Andrés Benavides Coral

18 de febrero de 2026

Resumen

Este documento introduce los conceptos fundamentales de `scikit-learn`, la biblioteca estándar de facto para Machine Learning clásico en Python. Diseñado para estudiantes universitarios con conocimientos de programación y álgebra lineal, el texto cubre la arquitectura uniforme de la biblioteca (estimadores, transformadores y pipelines), preprocesamiento, evaluación de modelos y selección de hiperparámetros. Se incluyen ejemplos reproducibles de clasificación, regresión, clustering y reducción de dimensionalidad, enfatizando en todo momento las buenas prácticas y la prevención de fugas de datos.

Índice

1. Introducción **3**

2. Instalación y Primeros Pasos **3**

3. Filosofía y Arquitectura **4**

4. Funciones y Clases Principales **4**
4.1. División de datos y Validación Cruzada 4

5. Casos de Uso **5**
5.1. Clasificación 5
5.2. Regresión 5
5.3. Clustering (Agrupamiento) 6
5.4. Reducción de Dimensionalidad 6

6. Preprocesamiento y Evaluación **7**

7. Buenas Prácticas: Pipelines **8**

8. Ejemplo Práctico Completo: Pipeline + GridSearchCV **8**

9. Resumen de Tareas y Algoritmos **9**

10. ¿Cuándo NO usar esta librería? **9**

11.Glosario **10**

12.Referencias y Recursos **10**

1. Introducción

`scikit-learn` es una biblioteca de código abierto para aprendizaje automático (Machine Learning) en Python. Construida sobre `NumPy`, `SciPy` y `Matplotlib`, fue iniciada como un proyecto de *Google Summer of Code* por David Cournapeau en 2007.

Su principal objetivo es ofrecer herramientas simples y eficientes para el análisis predictivo de datos. A diferencia de bibliotecas orientadas al Deep Learning (como PyTorch o TensorFlow), `scikit-learn` brilla en algoritmos clásicos, bases de datos tabulares y prototipado rápido, ofreciendo una de las APIs mejor diseñadas y más consistentes del ecosistema de software científico.

2. Instalación y Primeros Pasos

Para instalar la biblioteca, se recomienda usar el gestor de paquetes `pip` dentro de un entorno virtual:

```
# Instalación desde la terminal
pip install scikit - learn
```

Figura 1: Instalación de la biblioteca.

Una vez instalada, puedes verificar que funciona importándola en tu script o notebook. Nota que, aunque el paquete se instala como `scikit-learn`, se importa como `sklearn`.

```
# Verificar la versión instalada
import sklearn
print("Versión de scikit-learn:", sklearn.__version__)
```

Figura 2: Verificación de la instalación.

3. Filosofía y Arquitectura

La curva de aprendizaje de `scikit-learn` es suave gracias a su **API uniforme**. Todos los objetos comparten una interfaz predecible y consistente, dividida principalmente en tres abstracciones:

- **Estimators (Estimadores):** Cualquier objeto que pueda aprender de los datos. Todos implementan el método `fit(X, y)`.
- **Transformers (Transformadores):** Estimadores que también pueden modificar los datos. Implementan `transform(X)` o `fit_transform(X)`. Útiles para preprocesamiento.
- **Predictors (Predictores):** Modelos que, tras ser entrenados, pueden hacer predicciones sobre datos nuevos. Implementan `predict(X)` y, habitualmente, `score(X, y)` para evaluar el rendimiento.

4. Funciones y Clases Principales

Comprender los métodos base es crucial. Veamos un flujo de trabajo genérico:

```python
# 1. Instanciar el modelo (Fase de configuración)
from sklearn.linear_model import LogisticRegression
modelo = LogisticRegression(max_iter=1000)

# 2. Entrenar el modelo (Fase de aprendizaje: fit)
# X_train son las características, y_train las etiquetas
modelo.fit(X_train, y_train)

# 3. Predecir (Fase de inferencia: predict)
predicciones = modelo.predict(X_test)

# 4. Evaluar (M trica por defecto: score)
precision = modelo.score(X_test, y_test)
```

Figura 3: Ejemplo de flujo de trabajo genérico.

4.1. División de datos y Validación Cruzada

Para evitar el *overfitting* (sobreajuste, cuando el modelo memoriza pero no generaliza), dividimos los datos usando `train_test_split` o evaluamos mediante `cross_val_score` (entrena y evalúa en múltiples pliegues de los datos).

```python
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

# Cargar datos
X, y = load_iris(return_X_y=True)

# División simple (70% entrenamiento, 30% prueba)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, random_state=42)

# Validación cruzada (5 folds)
clf = DecisionTreeClassifier()
scores = cross_val_score(clf, X, y, cv=5)
print("Accuracy promedio:", scores.mean())
```

Figura 4: Ejemplo de división de datos y validación cruzada.

5. Casos de Uso

5.1. Clasificación

Teoría: Predecir una etiqueta categórica (discreta) para una observación. Por ejemplo, ¿es un correo Spam o No Spam?
Algoritmos típicos: `LogisticRegression`, `RandomForestClassifier`, `SVC`.

```python
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier

# Generar datos sinteticos para clasificación binaria
X, y = make_classification(n_samples=200, n_features=4, random_state=42)
X_train, X_test, y_train, y_test = train_test_split(X, y, random_state=42)

# Instanciar y entrenar
clf = RandomForestClassifier(n_estimators=50)
clf.fit(X_train, y_train)
print("Score clasification:", clf.score(X_test, y_test))
```

Figura 5: Uso de Clasificación.

5.2. Regresión

Teoría: Predecir un valor numérico continuo. Por ejemplo, predecir el precio de una casa en base a sus metros cuadrados.
Algoritmos típicos: `LinearRegression`, `RandomForestRegressor`, `SVR`.

```python
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression

# Datos sinteticos para regresión
X, y = make_regression(n_samples=100, n_features=1, noise=10, random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, random_state=42)

reg = LinearRegression()
reg.fit(X_tr, y_tr)
print("R^2 Score regresion:", reg.score(X_te, y_te))
```

Figura 6: Uso de Regresión.

5.3. Clustering (Agrupamiento)

Teoría: Aprendizaje no supervisado. Agrupar datos similares sin tener etiquetas previas.
Algoritmos típicos: `KMeans`, `DBSCAN`.

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Datos en 3 grupos naturales
X, _ = make_blobs(n_samples=300, centers=3, random_state=42)

# Agrupamiento
kmeans = KMeans(n_clusters=3, random_state=42)
# Solo usamos fit o fit_predict (no hay 'y')
etiquetas_predichas = kmeans.fit_predict(X)
```

Figura 7: Uso de Clustering.

5.4. Reducción de Dimensionalidad

Teoría: Reducir la cantidad de variables aleatorias a considerar, proyectando los datos en un espacio de menor dimensión, ideal para visualización o acelerar algoritmos.
Algoritmos típicos: `PCA`, `t-SNE`.

```python
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt

# Usamos los datos de Iris cargados anteriormente
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X) # Transforma de 4D a 2D

# Código para generar la gráfica
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='viridis', edgecolor='k')
plt.xlabel('Componente Principal 1')
plt.ylabel('Componente Principal 2')
plt.title('Visualización PCA de Iris')
plt.savefig('Figura_pca.png')
```

Figura 8: Uso de Reducción de Dimensionalidad.

Para incluir la figura generada en nuestro documento LaTeX, usaríamos el siguiente entorno:

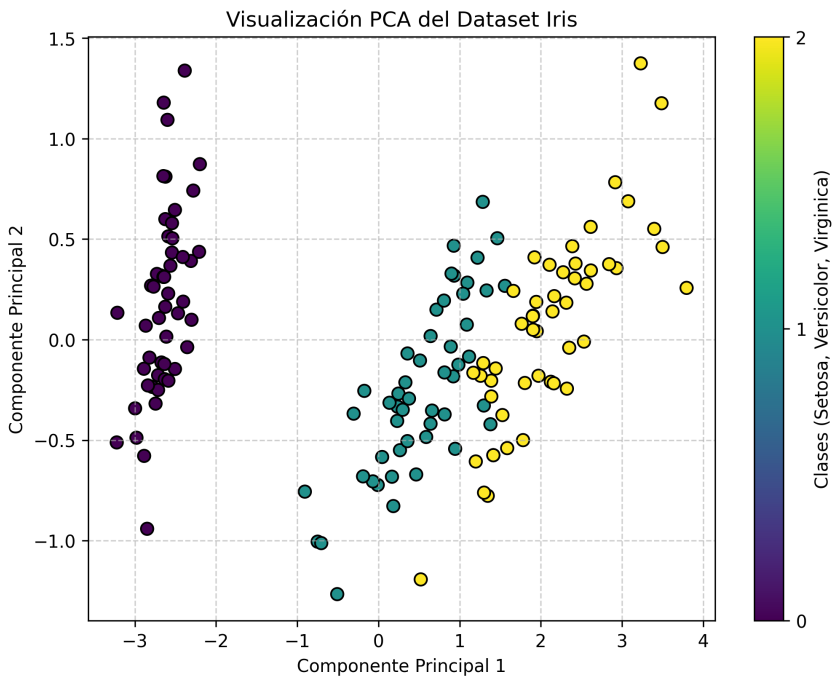

Figura 9: Proyección PCA del dataset Iris a 2 dimensiones.

6. Preprocesamiento y Evaluación

Casi ningún modelo funciona bien con datos crudos. `StandardScaler` estandariza las características eliminando la media y escalando a varianza unitaria.

En evaluación, no confíes solo en el `accuracy`. Dependiendo del problema, necesitas observar la matriz de confusión, precisión o error cuadrático.

```python
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # Importante: transform, no fit_transform

clf.fit(X_train_scaled, y_train)
y_pred = clf.predict(X_test_scaled)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred)) # Incluye precisión, recall y f1
```

Figura 10: Uso de StandardScaler.

7. Buenas Prácticas: Pipelines

El error más común en ML es la **fuga de datos (data leakage)**: usar información del conjunto de prueba durante el entrenamiento. Esto ocurre si aplicas `fit_transform` a todo el dataset *antes* de dividirlo. Para evitarlo, y para lograr un código limpio, usamos `Pipeline`.

```python
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC

# El Pipeline encadena transformadores y terminan en un estimador
tuberia = Pipeline([
  ('scaler', StandardScaler()),
  ('pca', PCA(n_components=3)),
  ('classifier', SVC())
])

# Al llamar fit, escala X_tr, aplica PCA, y entrena SVC automáticamente.
tuberia.fit(X_tr, y_tr)
```

Figura 11: Uso de Pipeline.

8. Ejemplo Práctico Completo: Pipeline + GridSearchCV

Un modelo rara vez tiene sus hiperparámetros óptimos por defecto. `GridSearchCV` prueba combinaciones de parámetros usando validación cruzada para encontrar el mejor modelo de forma segura.

```python
from sklearn.model_selection import GridSearchCV

# 1. Definir pipeline
pipe = Pipeline([
  ('scaler', StandardScaler()),
  ('clf', RandomForestClassifier(random_state=42))
])

# 2. Definir la malla de parametros a buscar
# Se usa el prefijo 'nombrePaso__' para acceder al parametro
param_grid = {
  'clf__n_estimators': [50, 100, 200],
  'clf__max_depth': [None, 5, 10]
}

# 3. Configurar GridSearch
grid_search = GridSearchCV(pipe, param_grid, cv=5, n_jobs=-1, scoring='accuracy')

# 4. Entrenar (esto prueba todas las combinaciones en los folds de CV)
grid_search.fit(X_train, y_train)

print("Mejores parametros:", grid_search.best_params_)
print("Score en test:", grid_search.score(X_test, y_test))
```

Figura 12: Ejemplo práctico Completo.

9. Resumen de Tareas y Algoritmos

Tarea	Clases Comunes	¿Cuándo elegirlas?
Clasificación	`LogisticRegression, SVC, RandomForest`	Predecir categorías (A, B, C).
Regresión	`LinearRegression, Ridge, SVR`	Predecir cantidades continuas.
Clustering	`KMeans, DBSCAN`	Descubrir grupos en datos no etiquetados
Red. Dim.	`PCA, TSNE`	Visualización de datos, evitar maldición

Tabla 1: Guía rápida de selección de algoritmos en `scikit-learn`.

10. ¿Cuándo NO usar esta librería?

A pesar de su versatilidad, `scikit-learn` tiene limitaciones:

- **Deep Learning:** No tiene soporte para redes neuronales profundas complejas ni aceleración por GPU. Para esto, usa PyTorch o TensorFlow.
- **Datos Masivos (Big Data):** Los algoritmos asumen que los datos caben en la memoria RAM (procesamiento *in-memory*). Para datasets masivos distribuidos, Spark MLlib es más adecuado.
- **Procesamiento de Lenguaje Natural Complejo:** Aunque tiene herramientas para extraer características de texto (`CountVectorizer`), carece de modelos de lenguaje modernos (Transformers). Para ello usa bibliotecas como `Hugging Face transformers`.

11. Glosario

API: Interfaz de Programación de Aplicaciones. Los métodos estándar para usar el código.

Fuga de datos (Leakage): Contaminar los datos de entrenamiento con información del conjunto de prueba.

Hiperparámetros: Configuraciones del algoritmo que se establecen antes del entrenamiento (ej. número de árboles en un bosque aleatorio) y no se aprenden de los datos directamente.

12. Referencias y Recursos

- Documentación oficial: https://scikit-learn.org
- Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.